

EDA II — Estructura de Datos y Algoritmos II

Documentación de Algoritmos

Universidad Nacional Autónoma de México
Facultad de Ingeniería

Proyecto	MiniRodri — Visualizador Interactivo
Equipo	Backend + Frontend + Chatbot IA
Materia	Estructura de Datos y Algoritmos II

1. Ordenación por Fusión — Merge Sort

Descripción

El **Merge Sort** es un algoritmo de ordenamiento basado en la técnica de *Divide y Vencerás*. Divide el arreglo recursivamente en mitades hasta obtener subarreglos de tamaño 1, luego los fusiona ordenadamente.

Complejidad

Caso	Tiempo	Espacio
Mejor	$O(n \log n)$	$O(n)$
Promedio	$O(n \log n)$	$O(n)$
Peor	$O(n \log n)$	$O(n)$

Implementación en Python

```
def merge_sort(arr, steps=None, comparisons=None): if steps is None: steps = [] if comparisons is None: comparisons = [0] if len(arr) <= 1: return arr mid = len(arr) // 2 izquierda = merge_sort(arr[:mid], steps, comparisons) derecha = merge_sort(arr[mid:], steps, comparisons) return _merge(izquierda, derecha, steps, comparisons) def _merge(izquierda, derecha, steps, comparisons): resultado = [] i = j = 0 while i < len(izquierda) and j < len(derecha): comparisons[0] += 1 if izquierda[i] <= derecha[j]: resultado.append(izquierda[i]); i += 1 else: resultado.append(derecha[j]); j += 1 resultado.extend(izquierda[i:]) resultado.extend(derecha[j:]) steps.append({"array": resultado.copy(), "descripcion": f"Merge de {izquierda} y {derecha}"}) return resultado
```

Endpoint del API

POST /api/sorting/merge-sort

Body: {"array": [5, 3, 8, 1, 2]}

Devuelve: sorted_array, steps, comparisons, complexity

2. Clasificación Rápida — Quick Sort

Descripción

El **Quick Sort** selecciona un elemento llamado *pivote* y particiona el arreglo en elementos menores y mayores al pivote. Aplica el proceso recursivamente a cada partición.

Complejidad

Caso	Tiempo	Espacio
Mejor	$O(n \log n)$	$O(\log n)$
Promedio	$O(n \log n)$	$O(\log n)$
Peor	$O(n^2)$	$O(n)$

Implementación en Python

```
def quick_sort(arr): arr = arr.copy() pasos = [] _quick_sort_recursivo(arr, 0, len(arr) - 1,
pasos) return { "algoritmo": "Quick Sort", "resultado": arr, "total_pasos": len(pasos), "pasos":
pasos } def _partition(arr, bajo, alto, pasos): pivote = arr[alto] i = bajo - 1 for j in
range(bajo, alto): if arr[j] <= pivote: i += 1 arr[i], arr[j] = arr[j], arr[i]
pasos.append({"arreglo": arr.copy(), "pivote": alto}) arr[i+1], arr[alto] = arr[alto], arr[i+1]
return i + 1
```

Endpoint del API

POST /api/sorting/quick-sort

Body: {"arreglo": [5, 3, 8, 1, 2]}

Devuelve: algoritmo, resultado, total_pasos, pasos

3. Árboles Binarios

Descripción

Un **árbol binario** es una estructura de datos jerárquica donde cada nodo tiene como máximo dos hijos: izquierdo y derecho. Se usa para búsquedas rápidas, ordenamiento y representación de expresiones matemáticas.

Operaciones principales

- **Inserción:** Agrega un nuevo nodo en la posición correcta.
- **Búsqueda:** Recorre el árbol comparando valores.
- **Recorridos:** Inorden, Preorden, Postorden.

Implementación en Python

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izquierdo = None
        self.derecho = None

class ArbolBinario:
    def __init__(self):
        self.raiz = None

    def insertar(self, valor):
        if self.raiz is None:
            self.raiz = Nodo(valor)
        else:
            self._insertar_recursivo(self.raiz, valor)

    def _insertar_recursivo(self, nodo, valor):
        if valor < nodo.valor:
            if nodo.izquierdo is None:
                nodo.izquierdo = Nodo(valor)
            else:
                self._insertar_recursivo(nodo.izquierdo, valor)
        else:
            if nodo.derecho is None:
                nodo.derecho = Nodo(valor)
            else:
                self._insertar_recursivo(nodo.derecho, valor)

    def inorden(self, nodo, resultado=None):
        if resultado is None:
            resultado = []
        if nodo:
            self.inorden(nodo.izquierdo, resultado)
            resultado.append(nodo.valor)
            self.inorden(nodo.derecho, resultado)
        return resultado
```

Notaciones

Notación	Descripción	Ejemplo (A+B*C)
Infija	Operador entre operandos	A + B * C
Prefija	Operador antes de operandos	+ A * B C
Sufija	Operador después de operandos	A B C * +

4. Estructuras de Grafos

Descripción

Un **grafo** es un conjunto de vértices (nodos) conectados mediante aristas. Se representan mediante *matriz de adyacencia* o *lista de adyacencia*. Los algoritmos principales son BFS (búsqueda por amplitud) y DFS (búsqueda en profundidad).

BFS — Búsqueda por Amplitud

```
from collections import deque def bfs(grafo, inicio): visitados = [] cola = deque([inicio]) vistos = set([inicio]) while cola: nodo = cola.popleft() visitados.append(nodo) for vecino in grafo[nodo]: if vecino not in vistos: vistos.add(vecino) cola.append(vecino) return visitados
```

DFS — Búsqueda en Profundidad

```
def dfs(grafo, inicio, visitados=None): if visitados is None: visitados = [] visitados.append(inicio) for vecino in grafo[inicio]: if vecino not in visitados: dfs(grafo, vecino, visitados) return visitados
```

Complejidad

Algoritmo	Tiempo	Espacio
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Matriz Adyacencia	$O(V^2)$ espacio	$O(V^2)$
Lista Adyacencia	$O(V + E)$ espacio	$O(V + E)$

Endpoints del API

POST /api/graphs/bfs — Recorrido por amplitud

POST /api/graphs/dfs — Recorrido en profundidad

POST /api/graphs/represent — Representaciones del grafo

Body: {"edges": [[0,1],[1,2]], "n": 3, "inicio": 0}

5. Complejidad Algorítmica

Descripción

La **notación Big O** describe el rendimiento de un algoritmo en función del tamaño de entrada n . Permite comparar algoritmos independientemente del hardware.

Tabla comparativa de algoritmos implementados

Algoritmo	Mejor	Promedio	Peor	Espacio
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$
Busq. Lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Busq. Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
BFS/DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$

Jerarquía de complejidades (de mejor a peor)

$O(1)$	Constante — Acceso directo
$O(\log n)$	Logarítmica — Búsqueda binaria
$O(n)$	Lineal — Búsqueda lineal
$O(n \log n)$	Linearítmica — Merge Sort, Heap Sort
$O(n^2)$	Cuadrática — Bubble Sort peor caso
$O(2^n)$	Exponencial — Evitar en lo posible